

MediKiosk

Complete Project Documentation

Pipelines + Materials + Why / How / Justification / Alternatives + Project Docs

AI Health Assistant with Voice, OCR Document Intelligence and Live Clinic Bar

Stack: Django 6.1.1 + SQLite + Ollama (llama3.2:1b) + Gemini 1.5-flash + Tesseract OCR + Web Speech API + Tailwind CSS

Generated from the live codebase. Backend: backend/. Frontend: frontend/templates + frontend/static.

Contents

1. Project overview (what, users, goals, safety boundaries)
2. System architecture (apps, request map, data stores)
3. End-to-end pipelines (chat, voice, OCR, live bar, documents)
4. Materials inventory (every technology with version and role)
5. Component deep-dives: why this, how used, justification, alternatives, could it be better
6. AI conversation design (state machine, intro-once, no-repeat, Hindi, suggestions)
7. Project documentation (features, API endpoints, data models, file map)
8. Setup, run and verification log
9. Known limits and roadmap

1. Project overview

MediKiosk is a clinic kiosk web application with two portals (Patient, Doctor), an AI health assistant chatbot with voice (speech in/out), OCR-based medical document understanding, and an always-visible Live clinic bar. It is an informational assistant, not a diagnostic or prescription system.

1.1 Users

- Patients: register/login, dashboard, medical history, medications, documents upload + OCR report, AI chatbot (text + live speech), history timeline, profile.
- Doctors: register/login, dashboard with patient list, patient detail (history, medications, documents), profile.
- Clinic operators: run the kiosk, use the Live bar queue indicator and Go-Live state.

1.2 Goals

- Understand typed or spoken concerns in English and Hindi (Devanagari + Roman Hinglish).
- Hold a polite, non-repeating conversation: introduce once, ask one gentle question at a time, never repeat intro/questions/paragraphs in a session.
- Give general self-care suggestions and general over-the-counter (OTC) educational info only when asked, always with a disclaimer, never dosage or prescription.
- Extract text and structure from uploaded medical documents (OCR) and show an aligned readable report.
- Keep a permanent per-session conversation record while sending only a sliding window to the LLM.

1.3 Safety boundaries (non-goals)

- No diagnosis, no prescription, no dosage. Chest pain / severe / red-flag symptoms always get prompt-care advice.
- All medicine content is general educational information with a pharmacist/doctor disclaimer.

2. System architecture

2.1 Django apps (backend/config/settings.py INSTALLED_APPS)

App	Role
config	Project settings, root URLconf, shared patient/doctor page views.
ai	Chat API + conversation engine (backend/ai/services.py). Endpoints under /api/ai/.
voice	Non-blocking voice chat API (/api/voice/chat/). Server-mic CLI loop kept only for local testing.
ocr	Document OCR pipeline (/ocr/). Tesseract + OpenCV + medical parsing + report.
patients / doctor / records / history	Domain data: patients, doctors, medical records, timelines.
abdm	Reserved integration area (health ID ecosystem).
live	Shared Go-Live state API (/api/live/) backing the bottom live bar.

2.2 Request map (backend/config/urls.py)

```
Browser (templates + static JS)
|-- /patient/... , /doctor/...   -> config/views.py (server-rendered pages)
|-- /api/ai/chat/ (POST question) -> ai/views.py ai_chat -> ai/services.ask_ai
```

```

|-- /api/voice/chat/ (POST text)      -> voice/views.py voice_chat -> ai/services.ask_ai
|-- /ocr/process/ (POST document)    -> ocr/views.py -> ocr_engine + data_extractor
|-- /api/live/status|toggle/         -> live/views.py (file-persisted live state)
|-- /api/patients/                   -> patients API (queue count for live bar)
`-- /static/...                       -> frontend/static (CSS/JS)
Ollama :8000? no - Ollama on 11434 (llama3.2:1b); Gemini cloud fallback.

```

2.3 Data stores

- SQLite db.sqlite3 (Django ORM: patients, doctors, records, documents). Simple, file-based, zero-ops for a kiosk.
- Per-session chat state: in-memory dict keyed by Django session_key (backend/ai/services.py _session_states).
- Permanent chat record: backend/media/chat_logs/<session>.jsonl (append-only, never truncated).
- Live state: backend/media/live_state.json (shared doctor <-> patient).
- Uploaded documents: backend/media/.

3. End-to-end pipelines

3.1 AI chat pipeline (POST /api/ai/chat/)

1. ai/views.ai_chat: parse question (form or JSON), resolve session_id (fixed bug: session.create() returns None, so key is captured after).
2. services.ask_ai(text, session_id) -> _ask_ai_internal:
 detect language (Devanagari/Hinglish/English) -> remember pain context
 -> exit-offer? -> medication-offer? -> direct medicine-request?
 -> long-chat check -> one-time short offer -> Ollama (fallback Gemini)
 -> intro-once, repetition guard, mark question asked, append history.
3. Response JSON: {question, answer, suggestions[3], otc_info, disclaimer}.

3.2 Voice pipeline (browser-first, responsive like v1)

Mic -> Web Speech API recognition (hi-IN/en-IN, Auto follows last message, manual Auto/EN/Hi toggle) -> POST text to /api/voice/chat/ (non-blocking)
 -> AI answer JSON -> speechSynthesis TTS (hi-IN/en-IN auto from answer)
 -> onend resumes listening. Server mic/whisper/pyttsx3 loop kept ONLY for local CLI testing (backend/test_voice.py), never in the web path.

3.3 OCR pipeline (POST /ocr/process/)

Upload -> save MedicalDocument -> ocr_engine.extract_document_data:
 preprocess variants (grayscale/contrast/binary/adaptive, 2x upscale)
 -> Tesseract image_to_string (best by valid-word score, not length)
 -> image_to_data word boxes (mapped back /UPSCALE_FACTOR)
 -> median-height row building -> per-row gap thresholds
 -> aligned-text reconstruction (4 spaces at column breaks)
 -> layout rows/groups/hierarchy + image size
 -> data_extractor.extract_medical_data(fields, labs, segments)
 -> store extracted_text/layout/segments/report -> JSON response.
 document_detail re-parses extracted_text so the report page never shows blank.

3.4 Live bar pipeline

base.html fixed bottom bar on every page -> main.js polls
 GET /api/live/status/ (3s) + GET /api/patients/ (queue, 20s)
 -> Go-Live circle POST /api/live/toggle/ {live, by} persists to media/live_state.json -> all tabs/doctors/patients sync green/red.

3.5 Documents pipeline

documents.html upload modal posts to ocr_process; document_detail.html renders patient info grid, CBC lab table, medical info, segments, raw OCR text and status. Upload toast lifted above the live bar.

4. Materials inventory

Material	Version / source	Role in project
Django	6.1.1 (pip)	Web framework: routing, templates, ORM, sessions, static.
SQLite	bundled, db.sqlite3	App database (patients, doctors, records, documents).
Ollama + llama3.2:1b	ollama pkg 0.6.2; model 1.2B Q8_0	Primary LLM, local, fast conversational answers.
Google Gemini	google-genai 2.22.0, gemini-1.5-flash	Fallback LLM when Ollama unavailable.
Web Speech API	browser native	Live STT (recognition) + TTS (synthesis), hi-IN/en-IN.
faster-whisper 1.2.1	pip (CLI only)	Local mic transcription for backend/test_voice.py.
SpeechRecognition 3.17.0 + pytsx3 2.99	pip (CLI only)	Local mic capture + offline TTS for CLI testing.
Tesseract OCR (pytesseract 0.3.13)	system binary + pip shim	Document text + word boxes.
OpenCV (cv2) + numpy	pip	Preprocessing: upscale, denoise, contrast, thresholds.
Pillow 12.3.0	pip	Image open/convert for OCR.
PyMuPDF 1.28.2	pip	PDF document support in OCR path.
Tailwind CSS (CDN)	cdn.tailwindcss.com	Utility styling in templates.
Vanilla JS	no framework	ai-chatbot.js, documents.js, main.js: chat, live speech, live bar.
python-dotenv	pip (settings import)	Loads .env (GEMINI_API_KEY etc.).
requests 2.34.2	pip	Test scripts + any server-side HTTP.
ngrok (+ ALLOWED_HOSTS/CSRF entries)	tunnel	Expose kiosk for remote/phone testing.
reportlab 5.0.1	pip (docs only)	Generated this PDF; not part of app runtime.

5. Component deep-dives: why / how / justification / alternatives / better?

5.1 Django 6.1 (web framework)

WHY this: WHY: batteries-included routing, templates, ORM, sessions, admin and static handling in one dependency; team knows Python; fastest path for a server-rendered kiosk.

HOW used here: HOW: config/urls.py maps pages + /api/*; config/views.py renders patient/doctor pages; ai/voice/ocr/live expose JSON APIs; Django sessions give per-user conversation isolation.

JUSTIFIED: JUSTIFIED: yes - avoids gluing Flask+SQLAlchemy+login libs; admin free; SQLite default matches kiosk scale.

ALTERNATIVES: ALTERNATIVES: Flask/FastAPI (lighter, but need extra auth/ORM/template wiring), Node/Express (JS everywhere, but team Python + AI libs are Python).

COULD IT BE BETTER: BETTER?: For multi-kiosk production: keep Django, add Gunicorn + PostgreSQL + Redis sessions; add DRF + OpenAPI for the JSON APIs.

5.2 SQLite (db.sqlite3)

WHY this: WHY: zero-config file DB, perfect for single-kiosk dev and demos; Django default.

HOW used here: HOW: DATABASES ENGINE django.db.backends.sqlite3; patients/doctors/records/documents models; medikiosk.db legacy file also present.

JUSTIFIED: JUSTIFIED: yes for now - single writer, tiny data, no ops burden.

ALTERNATIVES: ALTERNATIVES: PostgreSQL (concurrency, backups), MySQL (similar).

COULD IT BE BETTER: BETTER?: Migrate to PostgreSQL when >1 kiosk or concurrent writers; keep SQLite for offline kiosk image.

5.3 Django sessions for chat state

WHY this: WHY: fixes the critical bug where session.create() returns None and everyone shared one global conversation (intro missing, chest-pain hallucinated on 'Hello').

HOW used here: HOW: views capture session_key after create(); services keep _session_states[session_id] = SessionState (history window, asked set, flags); compat shims reset_conversation/reset_chat/reset_patient kept.

JUSTIFIED: JUSTIFIED: yes - minimal, no new infra, matches browser cookie flow.

ALTERNATIVES: ALTERNATIVES: JWT stateless (no server memory, but must ship history each call), Redis (best for scale).

COULD IT BE BETTER: BETTER?: Move SessionState to Redis with TTL + persist full_log already in JSONL; add explicit New-Chat endpoint instead of greeting heuristics.

5.4 Ollama llama3.2:1b (primary LLM)

WHY this: WHY: local inference = free, offline-capable, low latency (~0.2-1s observed), private patient text stays on machine; 1.2B Q8_0 fits modest hardware with 128k context.

HOW used here: HOW: ai/services.ask_ollama builds SYSTEM_PROMPT + per-state instruction + sliding history + lang directive; temperature 0.45, num_predict 80, keep_alive 10m; Gemini fallback on exception.

JUSTIFIED: JUSTIFIED: yes for kiosk triage chatter - fast and private; quality limits handled by deterministic guardrails (intro, no-repeat, suggestions).

ALTERNATIVES: ALTERNATIVES: bigger local models (llama3.1:8b, mistral) - smarter but slower/heavier; cloud-only (GPT/Gemini primary) - smarter but cost/latency/privacy hit.

COULD IT BE BETTER: BETTER?: Keep 1b for speed, add router: red-flag/complex -> bigger model or Gemini; add response caching for repeated FAQs.

5.5 Gemini 1.5-flash (fallback)

WHY this: WHY: keeps chat alive when Ollama is down; flash tier is fast/cheap; same prompt contract.

HOW used here: HOW: ask_gemini mirrors ask_ollama (history text, instruction, lang directive, polite post-process, intro). Model constant fixed from invalid gemini-3.6-flash.

JUSTIFIED: JUSTIFIED: yes as fallback only - primary stays local/private.

ALTERNATIVES: ALTERNATIVES: OpenAI GPT-4o-mini, Anthropic Haiku (similar role, different vendor).

COULD IT BE BETTER: BETTER?: Health-check Ollama at startup and show AI status in UI; retry with backoff.

5.6 Web Speech API (live voice in/out)

WHY this: WHY: the only responsive in-browser voice path: no audio upload, no server GPU, low latency; original server-mic design (mic on server hears nothing from the user's browser) cannot work on web.

HOW used here: HOW: ai-chatbot.js recognition (continuous+interim, hi-IN/en-IN, Auto follows last message + manual Auto/EN/Hi toggle) POSTs final text to /api/voice/chat/; answer spoken via speechSynthesis with matching voice/lang; onend resumes listening.

JUSTIFIED: JUSTIFIED: yes - restores the 'responsive voice like v1' requirement with zero server cost.

ALTERNATIVES: ALTERNATIVES: server Whisper on uploaded audio (accurate, but 1-5s latency + GPU/CPU load), cloud STT/TTS (Google/Azure - better Hindi voices, but cost + network + privacy).

COULD IT BE BETTER: BETTER?: Optional server-Whisper fallback for noisy audio; cloud neural TTS toggle for nicer Hindi voice; VAD tuning already via 500ms pause timer.

5.7 faster-whisper + SpeechRecognition + pyttsx3 (kept, CLI-only)

WHY this: WHY: useful for local Fitzgerald-style testing without a browser; pyttsx3 gives offline TTS on the dev machine.

HOW used here: HOW: voice/services.py listen_and_ask loop + backend/test_voice.py, test_tts.py; never imported by Django views (avoids loading Whisper weights into the web worker).

JUSTIFIED: JUSTIFIED: yes as dev/CLI harness - must stay out of the request path.

ALTERNATIVES: ALTERNATIVES: delete it (cleaner) or gate behind an env flag.

COULD IT BE BETTER: BETTER?: Keep but document clearly; add VOICE_CLI=1 guard so ``import voice.services`` never pulls models in web context.

5.8 Tesseract OCR + OpenCV + numpy + Pillow (+ PyMuPDF)

WHY this: WHY: free, offline document reading; OpenCV preprocessing rescues low-contrast scans; word boxes enable layout-aware reconstruction instead of flat text dumps.

HOW used here: HOW: ocr_engine tries grayscale/contrast/binary/adaptive variants at 2x, picks best by valid-word score, scales boxes back (/UPSCALE_FACTOR), median-height rows, per-row gap thresholds, 4-space column breaks; data_extractor parses fields/labs/segments; document_detail re-parses so report never blanks.

JUSTIFIED: JUSTIFIED: yes - commercial OCR APIs cost per page and leak patient documents.

ALTERNATIVES: ALTERNATIVES: EasyOCR/PaddleOCR (better handwriting, heavier), Google Vision/AWS Textract (best accuracy, cost + privacy).

COULD IT BE BETTER: BETTER?: Deskew + orientation fix; confidence-weighted voting; cache OCR JSON per file hash; async worker (Celery) so big PDFs never block the request.

5.9 Tailwind CDN + vanilla JS (frontend)

WHY this: WHY: fast utility styling without a build step; vanilla JS keeps kiosk pages light and debuggable.

HOW used here: HOW: base.html loads Tailwind + css/style.css; ai-chatbot.js owns chat/live-speech/TTS; documents.js owns upload modal + OCR result cards; main.js owns live bar polling/toggle.

JUSTIFIED: JUSTIFIED: yes for current scale - no SPA complexity needed.

ALTERNATIVES: ALTERNATIVES: React/Vue SPA (better state mgmt, but rewrite + build chain), Bootstrap (heavier, less flexible).

COULD IT BE BETTER: BETTER?: Bundle Tailwind locally for offline kiosk; split ai-chatbot.js into modules; add service worker for flaky clinic networks.

5.10 ngrok + hosts/CSRF entries

WHY this: WHY: instant public URL for phone/mic testing and demos without deploying.

HOW used here: HOW: ALLOWED_HOSTS/CSRF_TRUSTED_ORIGINS include ngrok domains; run ``ngrok http 8000``.

JUSTIFIED: JUSTIFIED: yes for dev/demo only.

ALTERNATIVES: ALTERNATIVES: Cloudflare Tunnel, Tailscale (more stable/secure).

COULD IT BE BETTER: BETTER?: Move to proper staging with HTTPS + secrets in env, DEBUG off, SECRET_KEY rotated (current key is committed in settings).

6. AI conversation design

6.1 State machine (per session)

```
fresh session -> intro ONCE (EN or Hindi per input language)
-> assessment questions (max 6), one gentle question/turn
-> ONE-TIME short offer: "I've taken your info. Now feel free to ask me anything."
    (Hindi: short Hindi equivalent)
-> yes -> suggestions + OTC info | no/ambiguous -> casual chat, NEVER re-offer
-> medicine keywords anytime -> suggestions + OTC info immediately
-> 20 turns -> polite exit offer once -> yes ends / no continues
```

6.2 Intro-once

Flag has_introduced set after the first assistant message; forced-intro runs only when history is empty. Fixed bug: previously the medication-offer paragraph re-fired every turn (screenshot case) because ambiguous replies cleared pending and the offer branch re-triggered; now medication_offered guarantees once-only.

6.3 No-repeat system

- asked_questions normalized set (never cleared mid-session) injected into the prompt as a NEVER-repeat list.
- recent_answers similarity check (exact or >85% word overlap) triggers one rephrase retry.
- collected_info (duration/severity/location/symptom, incl. Hindi keywords) injected as already-known facts.
- Post-assessment instruction tells the model to chat casually, never repeat offers/paragraphs.

6.4 Full history record (8-question cap removed)

Old code truncated history to 8-30 messages. Now each session keeps full_log (unbounded in memory) AND appends every turn to backend/media/chat_logs/<session>.jsonl. Only a 30-message sliding window is sent to the LLM for speed; get_full_log(session_id) returns the complete record.

6.5 Hindi: Devanagari + Hinglish, reply-only-matching

- Detection: Devanagari regex U+0900-U+097F OR Hinglish word-boundary markers (mujhe, dard, bukhar, dawa, kya, nahi...). Weak words (hai, kal, sir) need a second marker so 'hair fall'/'yes sir' stay English. Verified 10/10 incl. mid-session EN-HI-EN switching.
- Per-turn lang directive appended to Ollama+Gemini prompts; polite softening in both languages; Hindi suggestion/OTC tables; Hindi short offer; frontend TTS hi-IN/en-IN auto + mic Auto/EN/Hi toggle.

6.6 Medicine suggestions (safe)

Deterministic tables by pain type (headache/back/stomach/joint-muscle/tooth/throat/chest/general) + OTC educational note (paracetamol/ibuprofen commonly, label + pharmacist/doctor disclaimer, no dosage, no prescription). Chest pain always gets urgent-care advice, never OTC encouragement.

7. Project documentation

7.1 Features delivered

- Patient: landing, register/login/logout, dashboard, medical history, medications, documents upload + OCR report + delete, chatbot (text + live speech + TTS), timeline, profile.

- Doctor: landing, register/login, dashboard with patient search, patient detail (history/meds/documents/age/last visit), profile.
- Live bar on every page (base.html): clock, online status, role, queue count, red LIVE circle -> green when live, synced via /api/live/ across tabs and roles.
- Chat UX: welcome card, bubbles, image attach UI, recording popup, live overlay with wave + transcript + Auto/EN/Hi toggle, disclaimer footer.

7.2 API endpoints

Method + path	Purpose
POST /api/ai/chat/	Chat: {question text} -> {answer, suggestions[3], otc_info, disclaimer}.
POST /api/voice/chat/	Voice: {question text} -> same AI JSON (browser does STT/TTS).
POST /ocr/process/	Document OCR: {document} -> {text, medical_data, report, layout}.
GET /api/live/status/	Live state {live, since, by}.
POST /api/live/toggle/	Set live {live, by} -> persisted JSON.
GET /api/patients/	Patient list (queue count).
GET POST patient/*, doctor/*	Server-rendered pages + auth + CRUD.

7.3 Data models (essentials)

- Patient(name, dob, gender, blood_group, phone, email, address, password[hashed], patient_id PATxxxx).
- MedicalHistory(patient FK, condition, diagnosis, diagnosed_date, doctor_name, notes).
- Medication(patient FK, name, generic_name, dosage, frequency, route, dates, instructions, doctor).
- MedicalDocument(patient FK, document_name, file, document_type, extracted_text, ocr_layout/segments JSON, report).
- Doctor(full_name, reg. number unique, specialization, qualification, experience, phone, email, password[hashed], doctor_id DOCxxxx).

7.4 File map (where things live)

```

backend/config/      settings, urls, page views (views.py), wsgi/asgi
backend/ai/          services.py (engine), views.py (chat API), urls.py, prompts.py
backend/voice/       views.py (web API), services.py (CLI mic loop only), urls.py
backend/ocr/         ocr_engine.py, data_extractor.py, medical_parser.py, views.py
backend/patients|doctor|records|history|abdm|database|live
backend/media/       uploads + chat_logs/ + live_state.json
frontend/templates/  base.html, landing/, patient/* (note folder spelling), doctor/*
frontend/static/     css/style.css, css/ai-chatbot.css, js/main.js, js/ai-chatbot.js, ...

```

8. Setup, run and verification log

8.1 Setup & run

```

python -m venv venv
venv\Scripts\activate
pip install django ollama google-genai faster-whisper SpeechRecognition
pyttsx3 pytesseract pillow opencv-python numpy pymupdf requests python-dotenv
# Tesseract binary required on PATH for OCR.
python backend/manage.py migrate
python backend/manage.py runserver 127.0.0.1:8000
# AI needs Ollama running: ollama serve ; ollama pull llama3.2:1b

```

```
# .env: GEMINI_API_KEY=... (fallback only)
```

8.2 Verification evidence (from live runs)

Check	Result
manage.py check	0 issues.
py_compile services/views	OK.
POST /api/ai/chat/ Hello (fresh)	Intro once, polite, no ordering.
Same-session follow-ups	No intro repeat; varied questions; recorded info reused.
Medication offer loop (screenshot bug)	Fixed: short offer fires once; follow-ups never repeat it.
'What medicine can I take'	Suggestions + paracetamol/ibuprofen note returned.
Hindi Devanagari (5 msgs)	All replies Hindi.
Hinglish / EN / EN-HI-EN switch	All correct (10/10 incl. hair-fall/yes-sir negatives).
POST /api/voice/chat/	200 + intro + 3 suggestions.
GET /api/live/*	status/toggle sync across tabs.
test_ai.py	Passes: intro, assessment, reset_chat fresh patient.

9. Known limits and roadmap

9.1 Limits

- In-memory session states reset on server restart (JSONL logs survive, live reload not yet implemented).
- llama3.2:1b can hallucinate; guardrails (no-repeat lists, short offer, disclaimer) mitigate but do not eliminate.
- Roman-Hindi replies come back in Devanagari script (proper Hindi) by design.
- Browser Hindi TTS quality depends on installed voices; mic accuracy depends on Chrome/Edge + noise.
- SECRET_KEY committed, DEBUG=True, SQLite, no rate limiting: dev-only posture.

9.2 Roadmap (highest value first)

- Redis sessions + reload full_log on restart; explicit New-Chat button calling reset.
- Unicorn + PostgreSQL + env secrets + DEBUG off for any shared deployment.
- Model router (1b fast path, larger/cloud for red-flag or complex turns) + startup Ollama health check.
- OCR async worker + deskew + per-file-hash cache.
- Cloud neural TTS toggle for Hindi; server-Whisper fallback for noisy audio.
- Fix password field typo (migration), rename paitent/ folder, OpenAPI docs for JSON APIs.